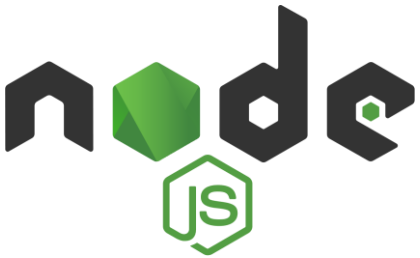
Activity12

Express

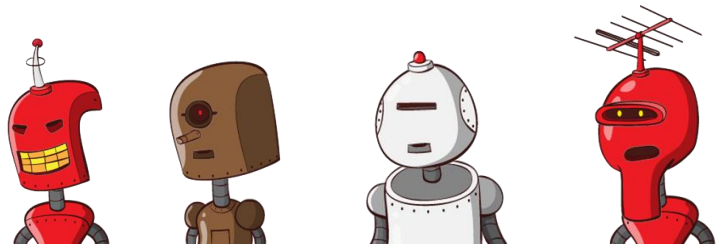


Part 1

Request Methods GET, POST,

UPDATE, DELETE

Part 2



Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.

This is the Part 2 of Activity 12.

In Part 1 we develop GET and POST
(13_1_20_Nodejs_Express_Request_Methods_GET_POST_DELETE_UPDATE).

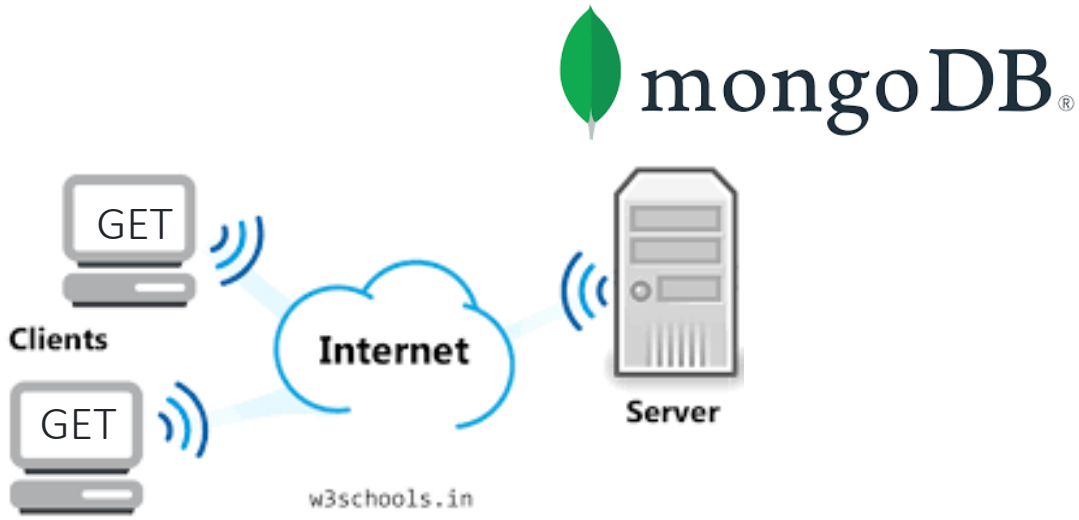
In Part 2 we develop UPDATE and DELETE operations.

Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.

Request Method Delete

```
app.delete("/deleteRobot/:id",
```

delete a Robot by its id



Observe the collection `robots`:

```
_id: ObjectId('69e5aec53ffd5c1cd874442b')
id : 3
name : "Robot 3"
price : "10000"
description : "This is a description of Robot 3"
imageUrl : "https://robohash.org/robot 3"
```

```
_id: ObjectId('69e5af4e3ffd5c1cd874442d')
id : "4"
name : "Robot 4"
price : "21.9"
description : "This is a description of Robot 4"
imageUrl : "https://robohash.org/robot 4"
```

Add delete Method using "/deleteRobot/:id"

```
app.delete("/deleteRobot/:id", async (req, res) => {
    try {
        const id = Number(req.params.id);

        await client.connect();
        console.log("Robot to delete :",id);

        const query = { id: id };

        // delete
        const results = await db.collection("robot").deleteOne(query);
        res.status(200);
        res.send(results);
    }
    catch (error){
        console.error("Error deleting robot:", error);
        res.status(500).send({ message: 'Internal Server Error' });
    }
});
```

In this example we are using the Robot Id to reference it and delete it.

Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.

We want to send back the information of the Robot just Deleted:

Before delete it:

```
// read data from robot to delete to send it to frontend
const robotDeleted = await db.collection("robot").findOne(query);
```

Send back the robot data :

```
res.send(robotDeleted);
```

```
// Delete Method

app.delete("/deleteRobot/:id", async (req, res) => {
  try {
    const id = Number(req.params.id);
    await client.connect();
    console.log("Robot to delete :", id);
    const query = { id: id };
    // read data from robot to delete to send it to frontend
    const robotDeleted = await db.collection("robot").findOne(query);
    if (robotDeleted) {
      // delete
      const results = await db.collection("robot").deleteOne(query);
      res.status(200);
      res.send(results);
    } else {
      res.status(404);
      res.send("Robot Not Found");
    }
  } catch (error) {
    console.error("Error deleting robot:", error);
    res.status(500).send({ message: "Internal Server Error" });
  }
});
```

Actually, we use descriptive end points, such as: /addRobots, /updateRobot, /deleteRobot :

HTTP Method		CRUD action
POST	/addRobots	Create a new robot
GET	/listRobots	Read a list of robot
GET	/{Id}	Read a single robot
PUT	/updateRobot/{id}	Update an existing robot
DELETE	/deleteRobot/{id}	Delete an existing robot

However, it is more appropriate as follows :

HTTP Method		CRUD action
POST	/robots	Create a new robot
GET	/robots	Read a list of robot
GET	/robots/{Id}	Read a single robot
PUT	/robots/{Id}	Update an existing robot
DELETE	/robots/{Id}	Delete an existing robot

Update HTML and Javascript in the frontend

Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.

DeleteRobot.jsx

```jsx
import React, { useState } from "react";

function DeleteRobots() {
  const [robotId, setRobotId] = useState("");

const handleChange = (event) => {
    setRobotId(event.target.value);
  };
const handleSubmit = async (event) => {
  event.preventDefault();
  const url = `http://127.0.0.1:8080/deleteRobot/${robotId}`;
  try {
    const response = await fetch(url, {
      method: "DELETE",
      headers: {
        "Content-Type": "application/json",
      },
    });
    if (!response.ok) {
      const errorText = await response.text();
      throw new Error(`Failed to delete Robot: ${errorText}`);
    }
    const result = await response.json();
    console.log("Success:", result);
    alert("Robot deleted successfully!");
    // Optionally clear the Robot ID input
    setRobotId("");
  } catch (error) {
    console.error("Error:", error);
    alert(`Error deleting Robot: ${error.message}`);
  }
};

return (
  <form onSubmit={handleSubmit}>
    <input
      type="text"
      value={robotId}
      onChange={handleChange}
      placeholder="Enter Robot ID"
      required
    />
    <br />
    <br />
    <button type="submit">Delete Robot</button>
  </form>
);
}
export default DeleteRobots;
```

Notice, we are using 127.0.0.1, which is the address for localhost

Here is the Robot Id

We want the delete the document, so set the method to DELETE

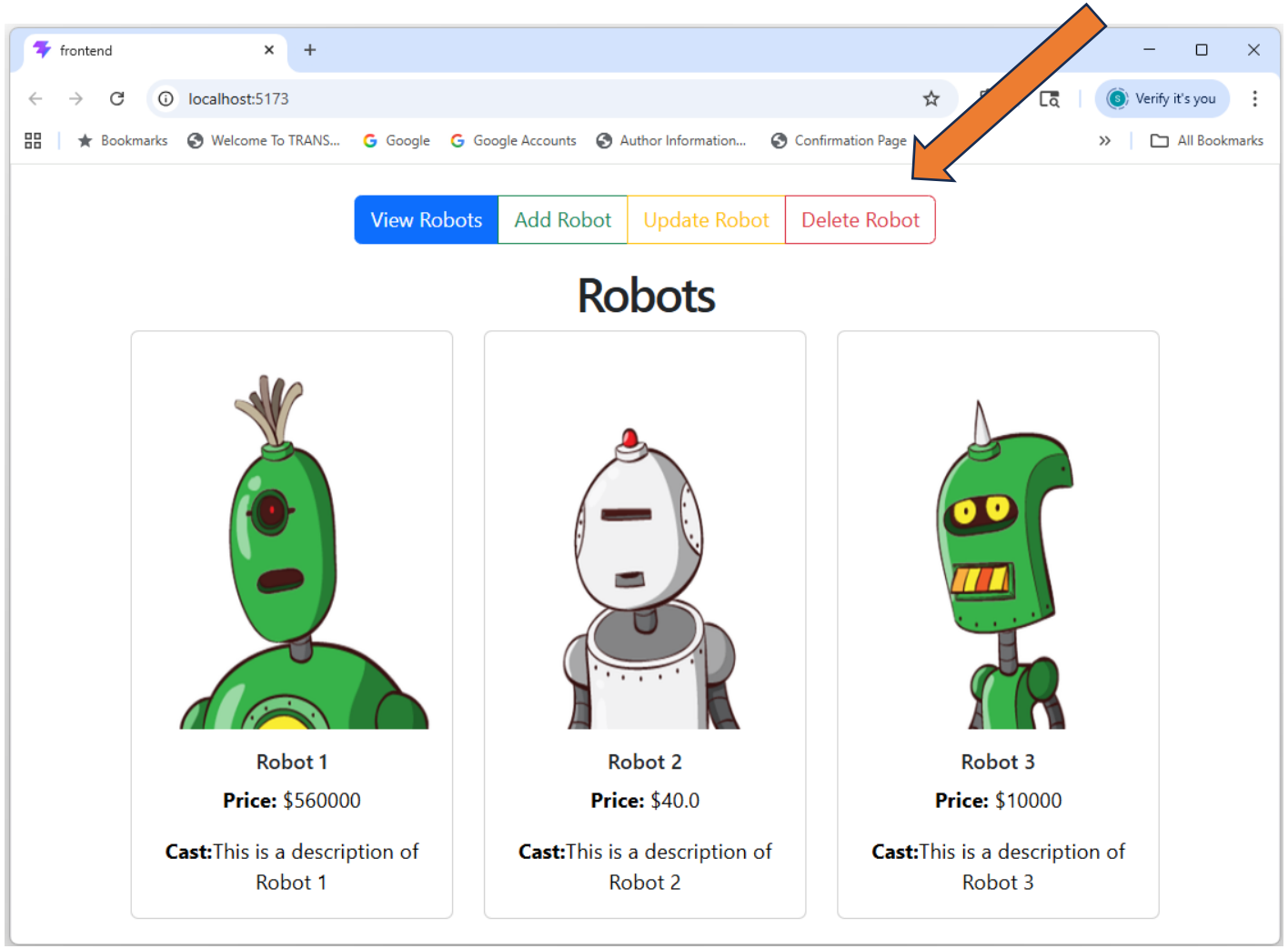
Once the robot is successfully deleted, we reset the form.

Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.

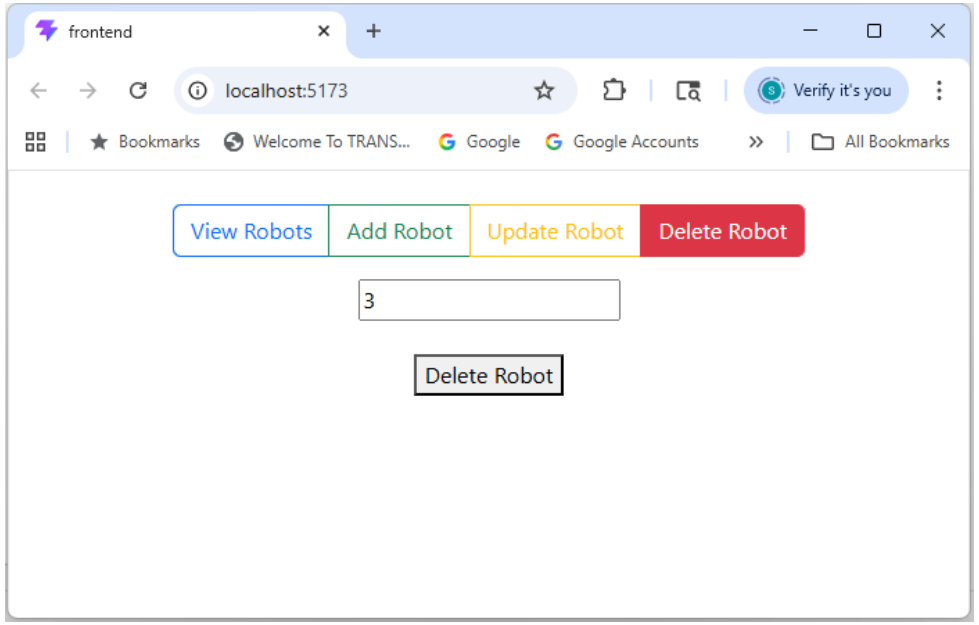
Lastly update the App.jsx to render the component upon the button click

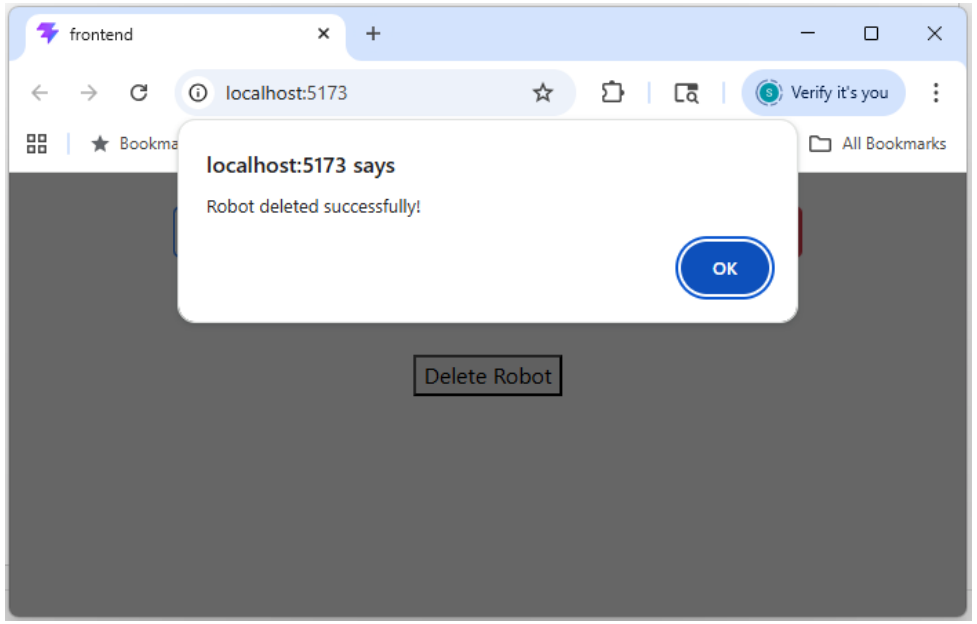
```jsx
import React, { useState } from "react";
import Robots from "./components/Robots";
import AddRobots from "./components/AddRobots";
import UpdateRobot from "./components/UpdateRobot";
import DeleteRobot from "./components/DeleteRobot";
import { Container, Row, Col, ButtonGroup, Button } from
"react-bootstrap";

const App = () => {
 const [activeComponent, setActiveComponent] =
useState("robots");

 return (
   <Container className="mt-4">
     <Row className="mb-3">
       <Col>
         <ButtonGroup>
           <Button
             variant={
               activeComponent === "robots" ? "primary" .
"outline-primary"
             }
             onClick={() => setActiveComponent("robots")}
           >
             View Robots
           </Button>
           <Button
             variant={
               activeComponent === "add" ? "success" : "outline-success"
             }
             onClick={() => setActiveComponent("add")}
           >
             Add Robot
           </Button>
           <Button
             variant={
               activeComponent === "update" ? "warning" : "outline-warning"
             }
             onClick={() => setActiveComponent("update")}
           >
             Update Robot
           </Button>
           <Button
             variant={
               activeComponent === "delete" ? "danger" : "outline-danger"
             }
             onClick={() => setActiveComponent("delete")}
           >
             Delete Robot
           </Button>
         </ButtonGroup>
       </Col>
     </Row>
     <Row>
       <Col>
         {activeComponent === "robots" && <Robots />}
         {activeComponent === "add" && <AddRobots />}
         {activeComponent === "update" && <UpdateRobot />}
         {activeComponent === "delete" && <DeleteRobot />}
       </Col>
     </Row>
   </Container>
 );
};

export default App;
```

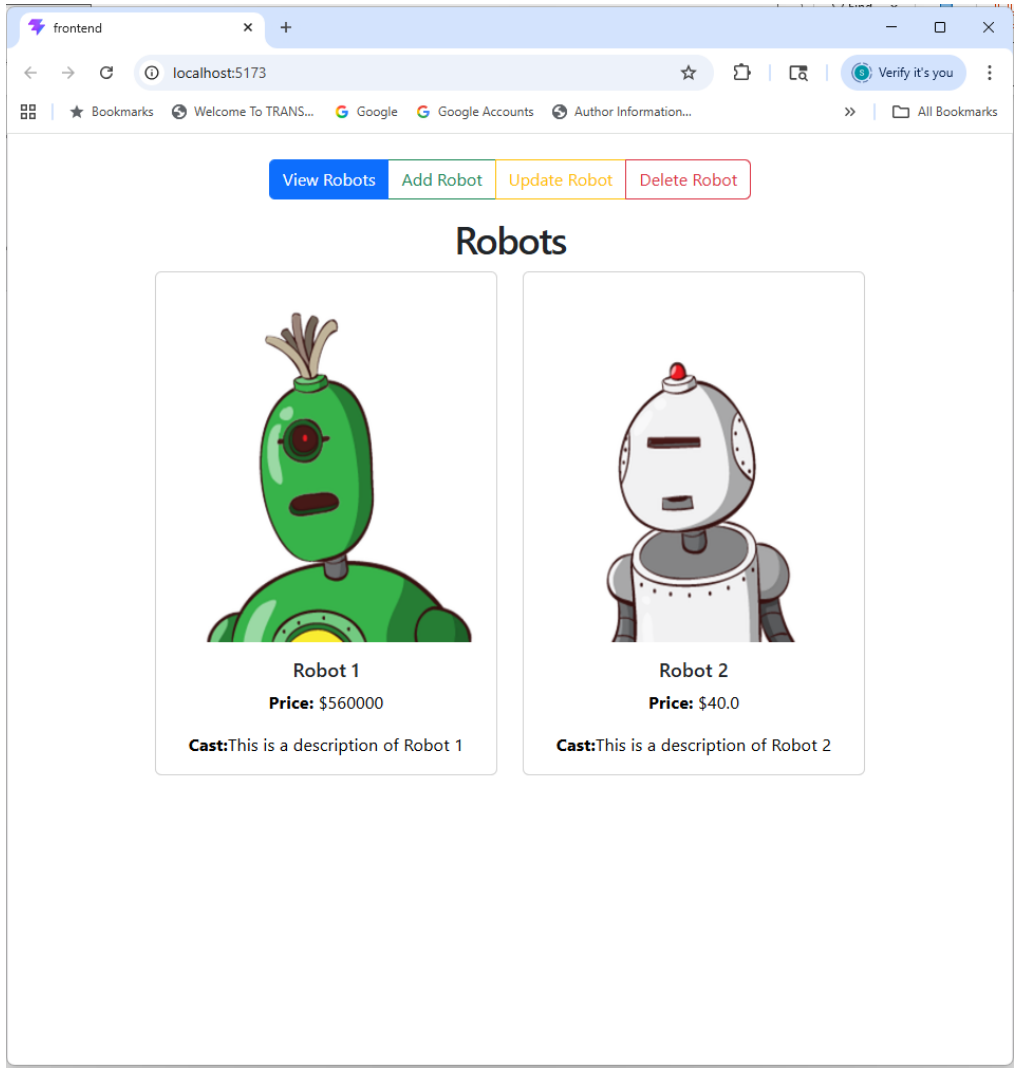
Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.



Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.





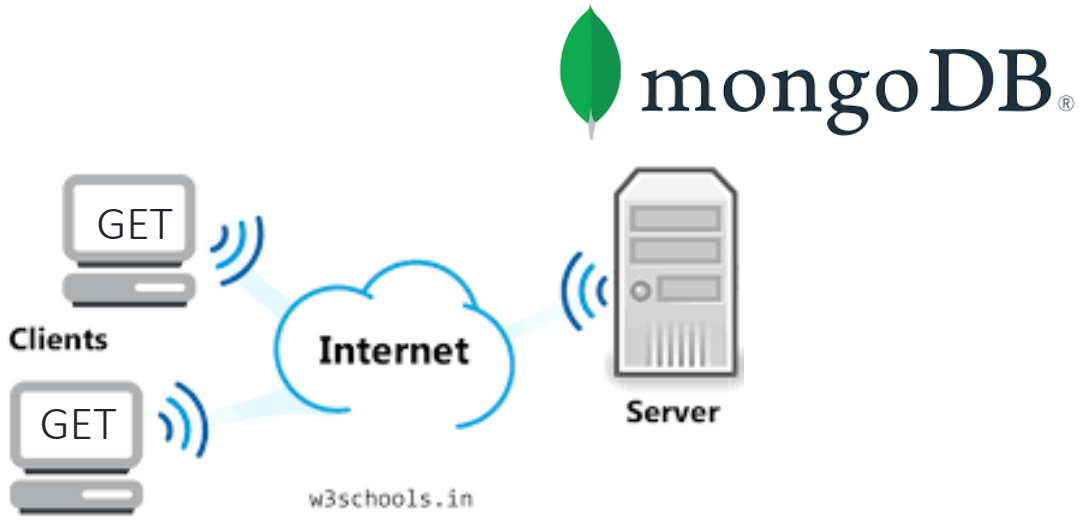


Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.

Request Method UPDATE

```
app.put("/updateRobot/:id",
```

update a Robot by its id



Add update Method using "/deleteRobot/:id"

```
app.put("/updateRobot/:id", async (req, res) => {
        const id = Number(req.params.id);
        const query = { id: id };

        await client.connect();
        console.log("Robot to Update :",id);

        // Data for updating the document, typically comes from the request body
        console.log(req.body);

        const updateData = {
            $set:{
                "name": req.body.name,
                "price": req.body.price,
                "description": req.body.description,
                "imageUrl": req.body.imageUrl
            }
        };

        // Add options if needed, for example { upsert: true } to create a document if it doesn't exist
        const options = { };
        const results = await db.collection("robot").updateOne(query, updateData, options);

        res.status(200);
        res.send(results);
});
```

We are using the Robot Id to reference it and update it.

We want to send back the information of the Robot just Updated:

Before update it:

```
// read data from robot to update to send to frontend
const robotUpdated = await db.collection("robot").findOne(query);
```

Send back the robot data :

```
res.send(robotUpdated);
```

If the robot you intend to update does not exist:

```
// If no document was found to update, you can choose to handle it by sending a 404
response
if (results.matchedCount === 0) {
    return res.status(404).send({ message: 'Robot not found' });
}
```

Update HTML and Javascript in the frontend

Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.

UpdateRobot.jsx

Copy all the code from AddRobots.jsx
We can use pretty much everything from that for PUT with just minor changes

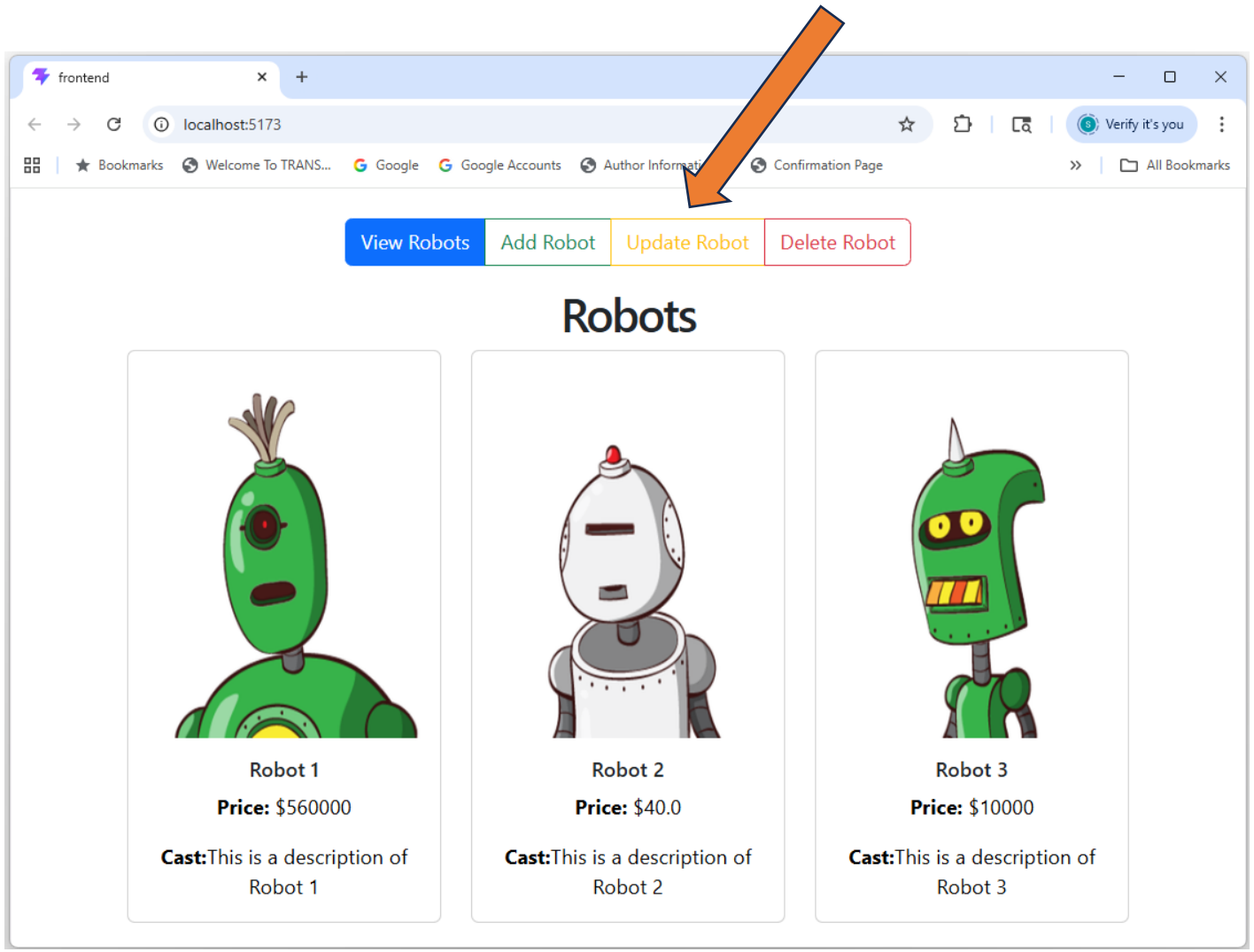
```jsx
import React, { useState } from "react";

const UpdateRobots = () => {
  const [formData, setFormData] = useState({
    id: "",
    name: "",
    price: "",
    description: "",
    imageUrl: "",
  });

  const handleChange = (event) => {
    const { name, value } = event.target;
    setFormData((prevState) => ({
      ...prevState,
      [name]: value,
    }));
  };
```

```jsx
const handleSubmit = async (event) => {
    event.preventDefault();
    const url = `http://127.0.0.1:8080/updateRobot/${formData.id}`;

      try {
      const response = await fetch(url, {
        method: "PUT",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify(formData),
      });

      if (!response.ok) {
        throw new Error(`Failed to update Robot: ${errorText}`);
      }
      const result = await response.json();
      console.log("Success:", result);
      alert("Robot updated successfully!");

      // Optionally reset form or navigate away
      setFormData({
        id: "",
        name: "",
        price: "",
        description: "",
        imageUrl: "",
      });
    } catch (error) {
      console.error("Error:", error);
      alert(`Error updating Robot: ${error.message}`);
    }
  };
```

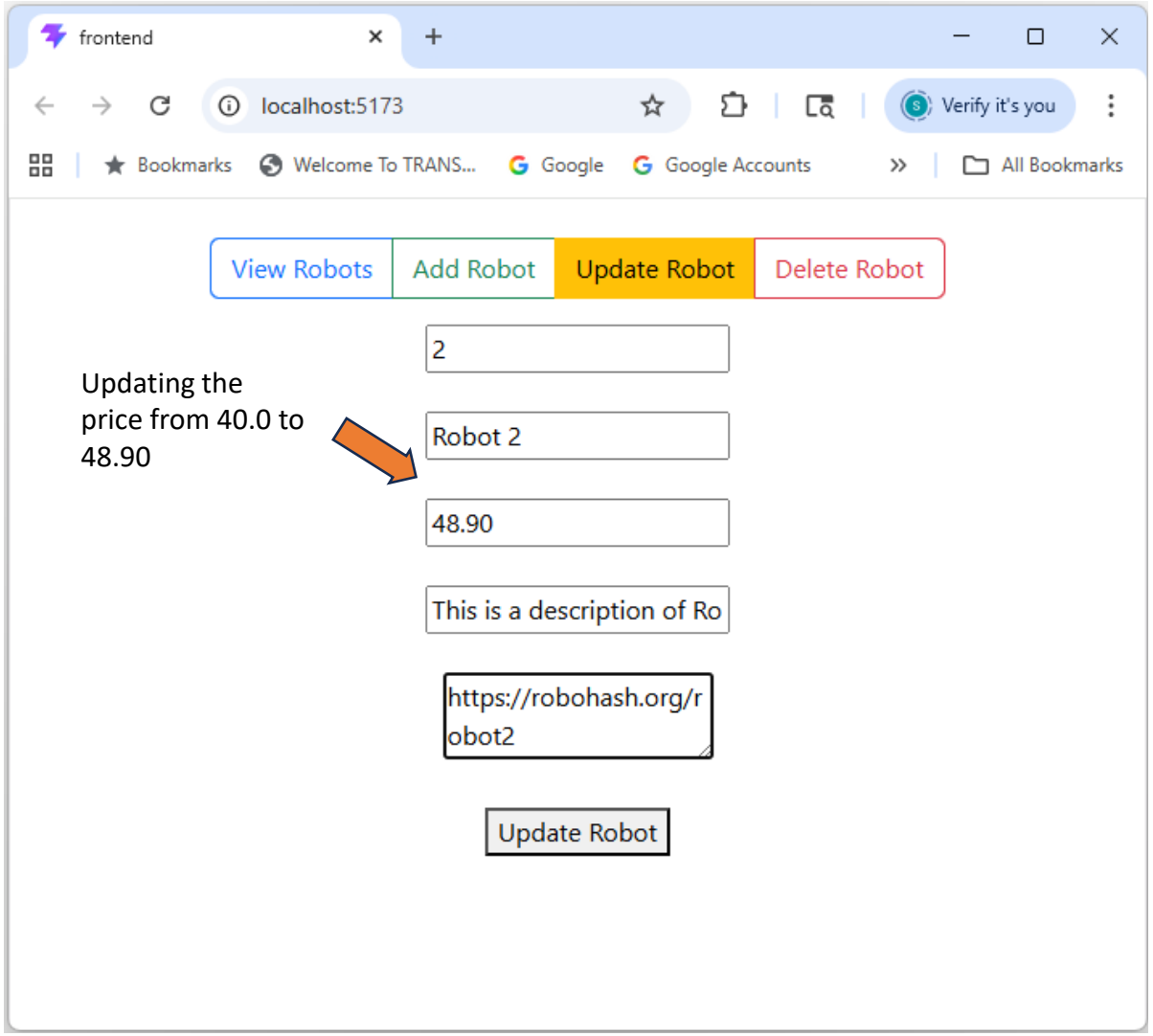
Fix the error messages as per the update robot

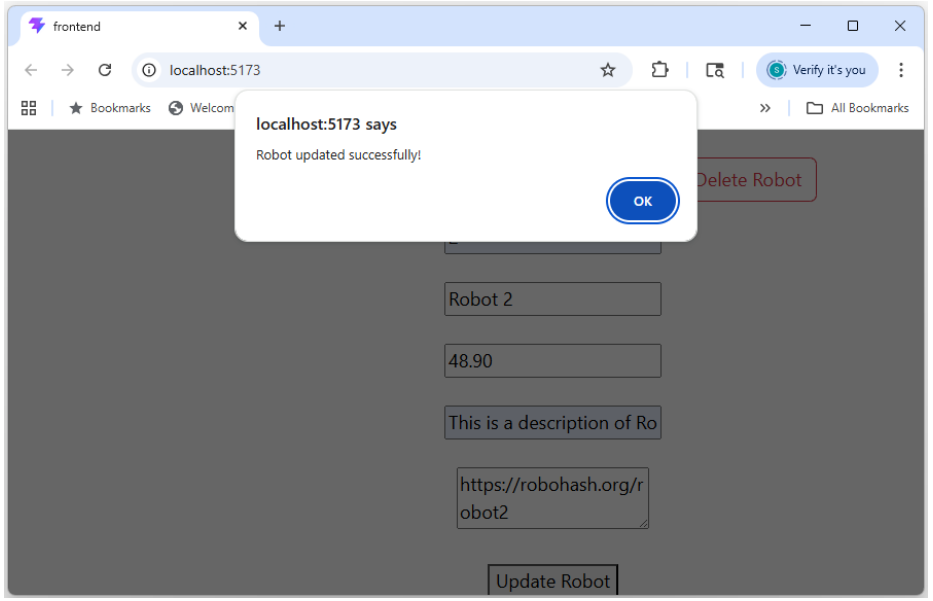
Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.

```
return (
    <>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          name="id"
          value={formData.id}
          onChange={handleChange}
          placeholder="ID"
          required
        />{" "}
        <br />
        <br />
        <input
          type="text"
          name="name"
          value={formData.name}
          onChange={handleChange}
          placeholder="Robot Name"
          required
        />
        <br />
        <br />
        <input
          type="text"
          name="price"
          value={formData.price}
          onChange={handleChange}
          placeholder="Robot Price"
          required
        />
        <br />
        <br />
        <input
          type="text"
          name="description"
          value={formData.description}
          onChange={handleChange}
          placeholder="Robot Description"
          required
        />
        <br />
        <br />
        <textarea
          name="imageUrl"
          value={formData.imageUrl}
          onChange={handleChange}
          placeholder="Robot Image URL"
          required
        />
        <br />
        <br />
        <button type="submit">Update Robot</button>
      </form>
    </>
  );
};

export default UpdateRobots;
```

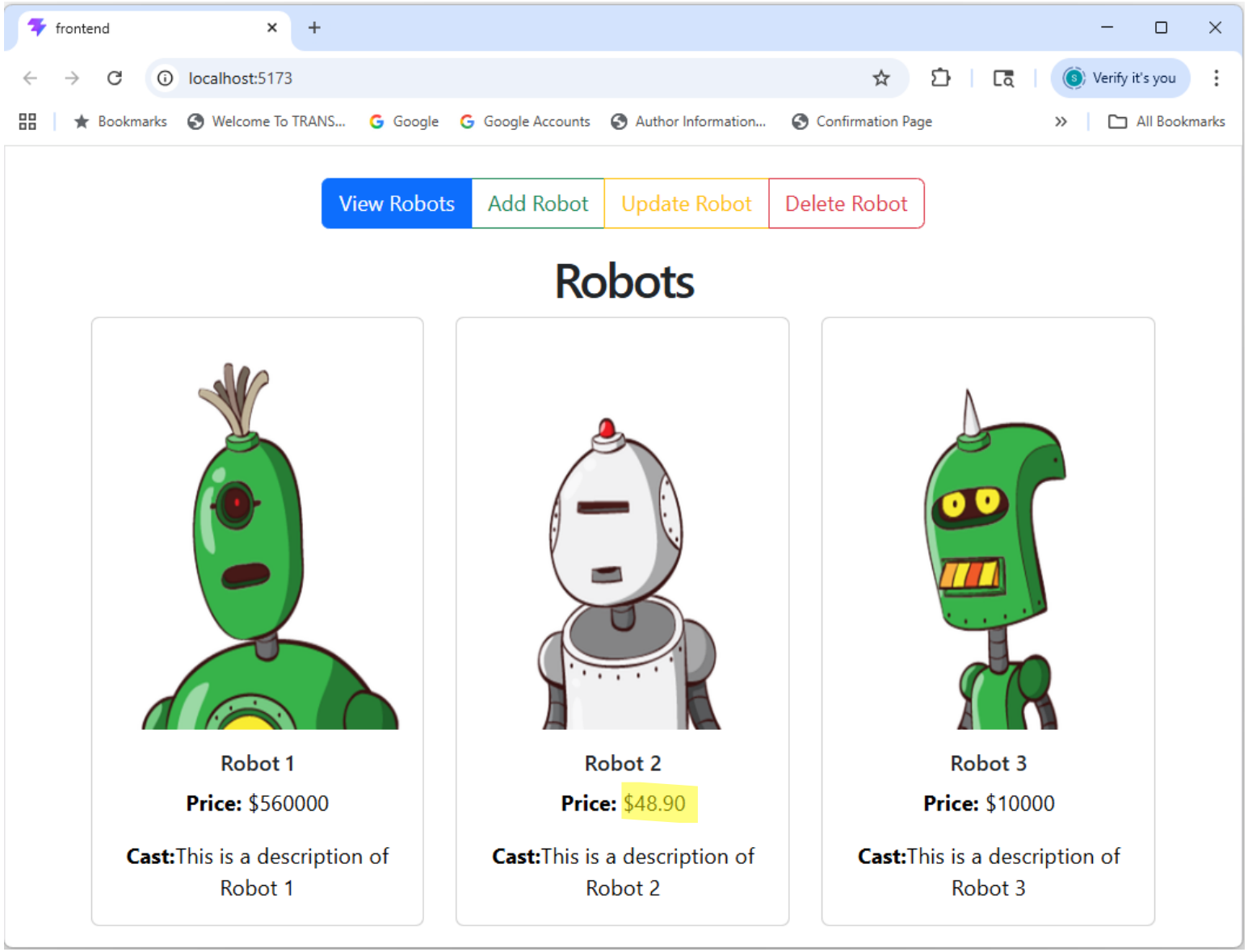


Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.





Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.



Sarfaraz Ahmed Mohammed, Ph.D.
Ali Jannesari, Ph.D.

We have the API

working

GET

POST

PUT

DELETE